
A QWIK GUIDE

Scaling Numeric Data for Machine Learning

A beginner-friendly walk through why models care about scale — and how standardization, min-max, robust, and log scaling put every feature on equal footing.

StandardScaler

MinMaxScaler

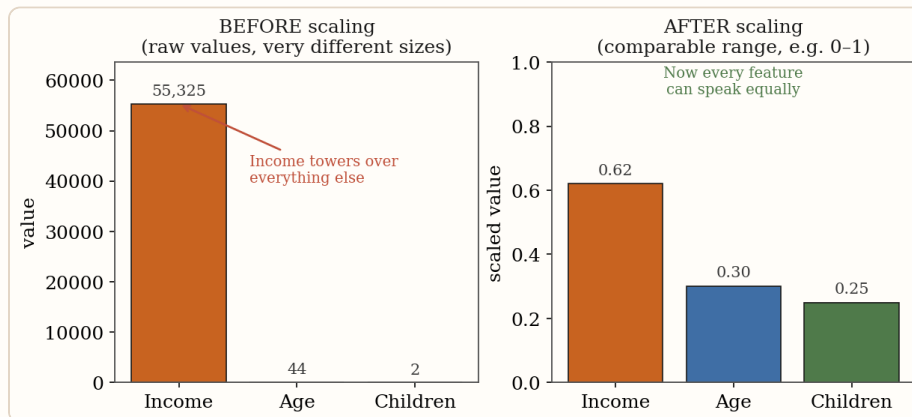
Robust & Log

Frequency Encoding

KNN + Metrics

1 Why Scale Numeric Features?

Imagine a dataset with **Income** (tens of thousands) sitting next to **Age** (tens) and **Children** (single digits). To many machine-learning algorithms, the biggest numbers look like the most important ones — purely because they are big. **Feature scaling** fixes this by transforming numeric features onto a common scale, so no single feature dominates just because of its units.



Before scaling, Income drowns out the other features. After scaling, each one contributes on equal terms.

What goes wrong if you skip it

- **Biased models.** Features with larger magnitudes dominate learning, giving certain inputs unfair weight and hurting predictions.
- **Slow or failed training.** Neural networks trained with gradient descent can converge slowly — or oscillate and never converge — when feature scales are uneven.
- **Unstable maths.** Algorithms built on distance calculations or matrix operations can hit overflow or underflow, hurting accuracy and reliability.
- **Harder to interpret.** When every feature is in different units, it is difficult to compare their relative importance.

KEY IDEA

Scaling removes bias, speeds up training, stabilizes the maths, and makes models easier to read — so **every feature contributes more equally** to what the model learns.

Some algorithms don't just *benefit* from scaling — they effectively **require** it. Support vector machines, principal component analysis (PCA), Lasso and Ridge regression, neural networks, and distance-based methods like k-nearest neighbors all expect scaled inputs. (Tree-based models are the usual exception — they generally don't need scaling.)

2 The Four Main Scaling Techniques

There are many scaling methods (MaxAbs scaling, power transforms, unit-vector normalization, and more), but four cover the vast majority of real work. Here they are at a glance:

Technique	Best for	Outliers	Typical algorithms
Standardization (Z-score)	Normally distributed data	Sensitive	SVM, PCA, logistic regression
Min-max scaling	Bounded or skewed data (keep the shape)	Very sensitive	Neural networks, KNN
Robust scaling	Non-Gaussian / skewed data	Handles them well	Scale-sensitive models when outliers are present
Log scaling	Exponential / right-skewed data	Pulls them inward	Financial data, feature engineering

GOOD TO KNOW

Standardization, min-max, and robust scaling all **preserve the shape** of your data's distribution — they just reposition and rescale it. Log scaling is the odd one out: it deliberately **changes the shape** to reduce skew.

A NOTE ON TREE-BASED MODELS

You'll sometimes see robust scaling paired with tree-based models and gradient boosting. In practice, **most tree-based models don't need scaling at all** — they split on one feature at a time, so a feature's raw magnitude doesn't matter. Robust scaling becomes relevant for them only inside a mixed pipeline that also feeds a scale-sensitive step. If you're using trees alone, you can usually skip scaling entirely.

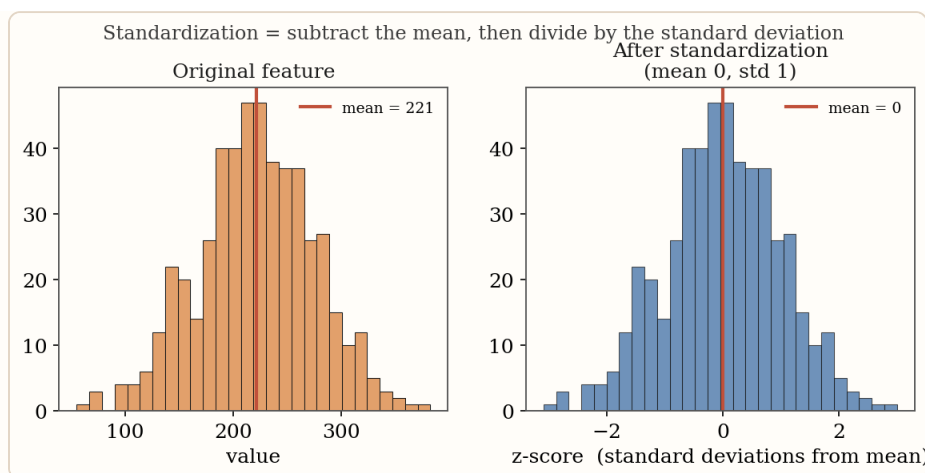
3 Standardization (Z-Score Scaling)

Standardization rescales a feature so it has a **mean of 0** and a **standard deviation of 1** (which is the same as unit variance). It works in two steps — **centering** then **scaling**:

THE FORMULA

$$z = (x - \text{mean}) / \text{standard deviation}$$

Subtracting the mean centers the data around zero; dividing by the standard deviation gives it unit variance. The result, **z**, is a **z-score** — how many standard deviations a value sits above or below the mean.



Standardization slides the distribution so its center is 0 and stretches it to a standard width of 1 — the shape is unchanged.

Worked example

Suppose **Feature A** has values 150, 200, 250, 300 (mean 225, standard deviation 55.9) and **Feature B** has 20, 30, 40, 50 (mean 35, standard deviation 11.18). Standardizing the smallest value of each:

- Feature A: $(150 - 225) / 55.9 \approx -1.34$
- Feature B: $(20 - 35) / 11.18 \approx -1.34$

Both land on the same z-score of **-1.34**. The **minus sign** tells us each value sits about 1.34 standard deviations *below* its own average — so they're now directly comparable even though their original units were wildly different.

When to use it — and when not to

Reach for standardization when...	Avoid it when...
Data is roughly bell-shaped (heights, weights, test scores)	Data is heavily skewed or has multiple peaks (multimodal)
The algorithm assumes normality (logistic regression, LDA, Gaussian Naive Bayes)	There are many outliers — they distort the mean and standard deviation

WATCH OUT

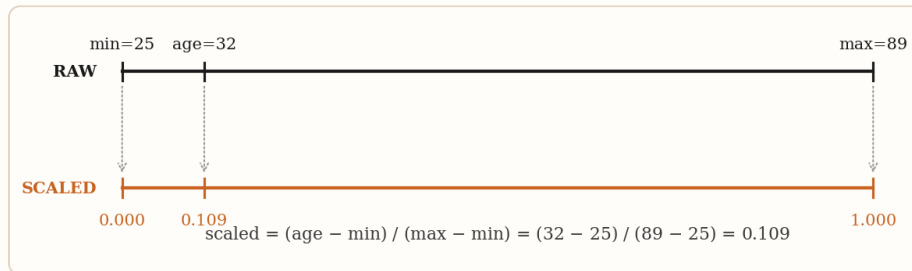
Because the mean and standard deviation are both pulled around by extreme values, standardization is a poor choice when your data is full of **outliers**. In that case, use robust scaling (Section 5).

4 Min-Max Scaling

Min-max scaling squeezes every value into a fixed range — usually **0 to 1**. The smallest value becomes 0, the largest becomes 1, and everything else lands proportionally in between.

THE FORMULA

$$\text{scaled} = (x - \min) / (\max - \min)$$



With Age running 25–89, an age of 32 maps to $(32-25)/(89-25) = 0.109$ on the 0–1 scale.

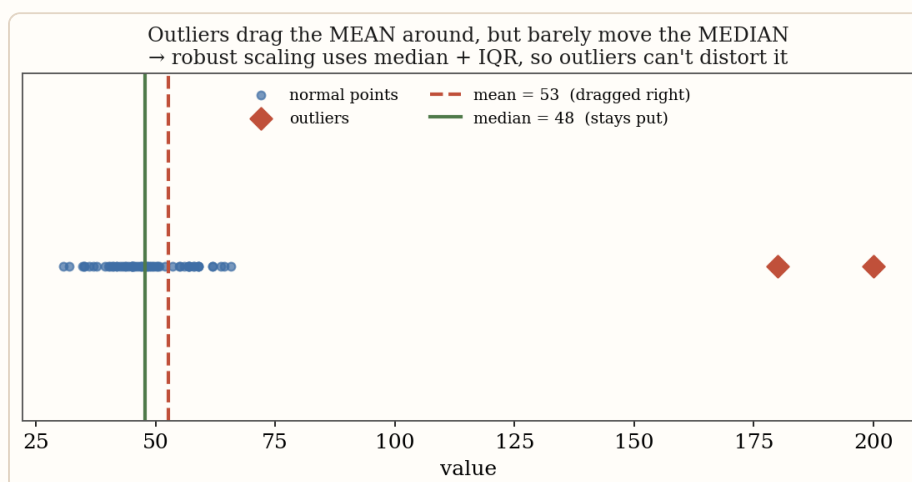
Min-max scaling is a natural fit when the data has known upper and lower bounds, or when an algorithm needs inputs in a specific range — neural networks and KNN are common examples. Like standardization, it **preserves the shape** of the distribution.

THE BIG WEAKNESS

Min-max scaling is **extremely sensitive to outliers**. A single extreme value sets the min or max, which then squashes every other point into a tiny sliver of the range. If outliers are present, prefer robust scaling.

5 Robust Scaling

Robust scaling is standardization's outlier-proof cousin. Instead of the mean and standard deviation, it centers with the **median** (the 50th percentile) and scales with the **interquartile range** — the IQR, which is the 75th percentile minus the 25th.



Two outliers drag the mean far to the right, but the median barely moves — which is exactly why robust scaling shrugs off extreme values.

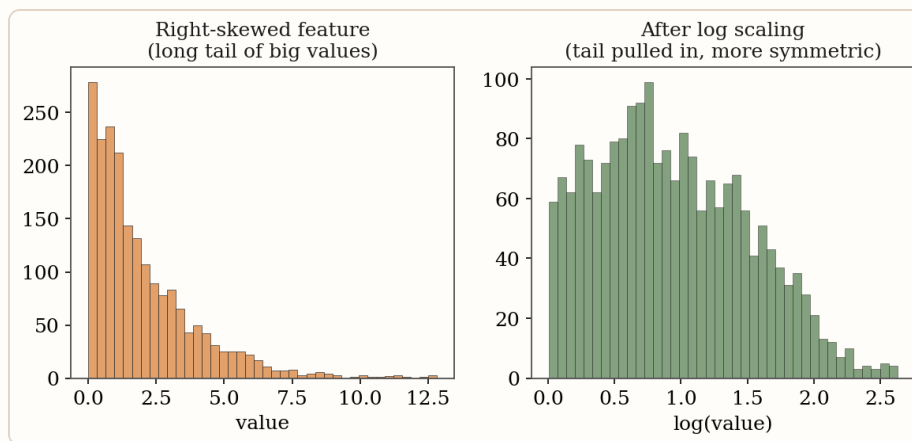
Why does this work? The median and IQR describe the *middle* of the data, and outliers — by definition — lie beyond the interquartile range. So they never leak into the scaling parameters. Robust scaling also **keeps the distribution's shape** and makes *no* assumption of normality, which makes it ideal for non-Gaussian, skewed datasets.

RULE OF THUMB

Skewed data with outliers, and you want to keep the shape intact? **Robust scaling** is your default.

6 Log Scaling

The first three techniques preserve shape. Log scaling is the one that deliberately changes it. By applying a logarithm, it **compresses large values and spreads out small ones** — taming wide ranges and reducing right skew.

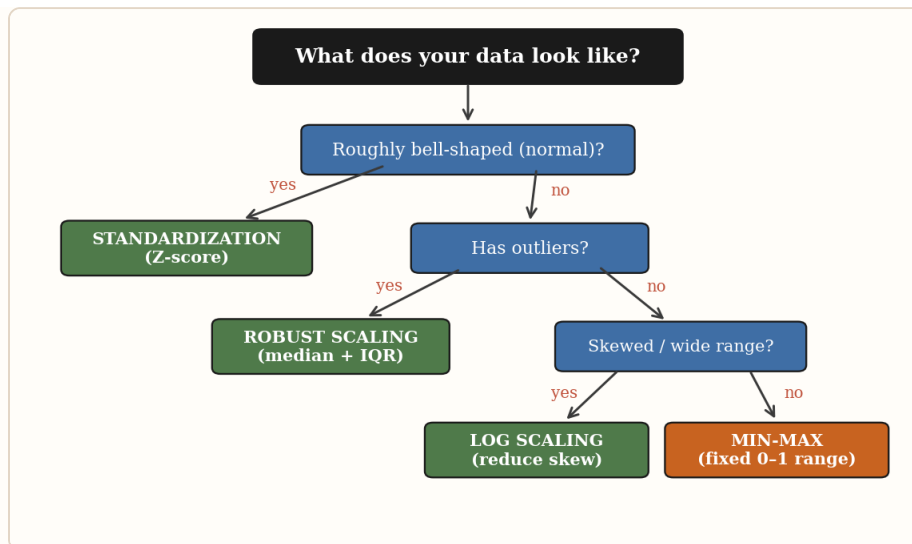


A long right tail before log scaling becomes far more symmetric afterwards.

- **Reduces skew.** The output is less skewed than the input — handy when a model prefers symmetric data.
- **Softens outliers.** Big values get pulled in toward the center, lessening their influence.
- **Can straighten relationships.** If a variable grows exponentially (like e^x), taking its log recovers a linear relationship — a gift for linear regression and correlation.

7 Choosing the Right Method

Three questions usually settle it: **What shape is the data? Are there outliers? Which algorithm are you feeding?**



A quick decision path. When in doubt on bell-shaped data, standardization is a safe default; for outliers, reach for robust scaling.

8 The Golden Rule: Fit on Training Data Only

This is the single most important habit in this guide. You **fit** a scaler (learn its min, max, mean, or standard deviation) using the **training data only**. You then **transform** the test data using those same learned values — never re-fitting on the test set.

WHY IT MATTERS

Letting test-set information influence the scaler is called **data leakage**. It quietly inflates your scores and produces a model that looks great in testing but disappoints in the real world. Fitting on test data would be cheating.

```
# The toolkit we'll use
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, roc_auc_score)
```

```
# 1) Split FIRST – before any scaling
X = buyers_df.drop(columns='Purchased Bike')
y = buyers_df['Purchased Bike']

X_train_df, X_test_df, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
# 2) Fit the scaler on TRAINING data only
scaler = MinMaxScaler(feature_range=(0, 1))
X_train_df['Age_scaled'] = scaler.fit_transform(X_train_df[['Age']])

# Age min=25, max=89 -> age 32 becomes:
# (32 - 25) / (89 - 25) = 0.109

# 3) Re-use the SAME scaler on the test data (transform only)
X_test_df['Age_scaled'] = scaler.transform(X_test_df[['Age']])
```

Notice the pattern: `fit_transform` on train, then `transform` on test. Standardization follows the exact same rhythm:

```
scaler = StandardScaler()
X_train_df['Income_scaled'] = scaler.fit_transform(X_train_df[['Income']])

# Income mean=55325, std=30940.9 -> income 20000 becomes:
# (20000 - 55325) / 30940.9 = -1.14 (a z-score)

X_test_df['Income_scaled'] = scaler.transform(X_test_df[['Income']])
```

9 Frequency Encoding for Categorical Data

Scaling handles numbers, but models also need categories (like `Region`) turned into numbers. **Frequency encoding** replaces each category label with how often it appears — its relative frequency in the training data. For example, in this dataset "North America" appears in 402 of the 800 training rows, so it becomes **0.5025** ($402 \div 800$).

Below is a small custom transformer. By inheriting from scikit-learn's `BaseEstimator` and `TransformerMixin`, it gains `fit_transform` for free and slots neatly into any pipeline:


```

from sklearn.base import BaseEstimator, TransformerMixin

class FrequencyEncoder(BaseEstimator, TransformerMixin):
    def __init__(self, cols=None):
        self.cols = cols

    def fit(self, X, y=None):
        self.freqs_ = {}
        for col in self.cols:
            # learn each category's share of the rows
            self.freqs_[col] = X[col].value_counts(normalize=True)
        return self

    def transform(self, X):
        X_transformed = X.copy()
        for col in self.cols:
            X_transformed[col] = X_transformed[col].map(self.freqs_[col])
        return X_transformed

# 'North America' (402 of 800 rows) -> 0.5025
freq_encoder = FrequencyEncoder(cols=categorical_cols)
X_train_encoded = freq_encoder.fit_transform(X_train_df)
X_test_encoded = freq_encoder.transform(X_test_df)

```

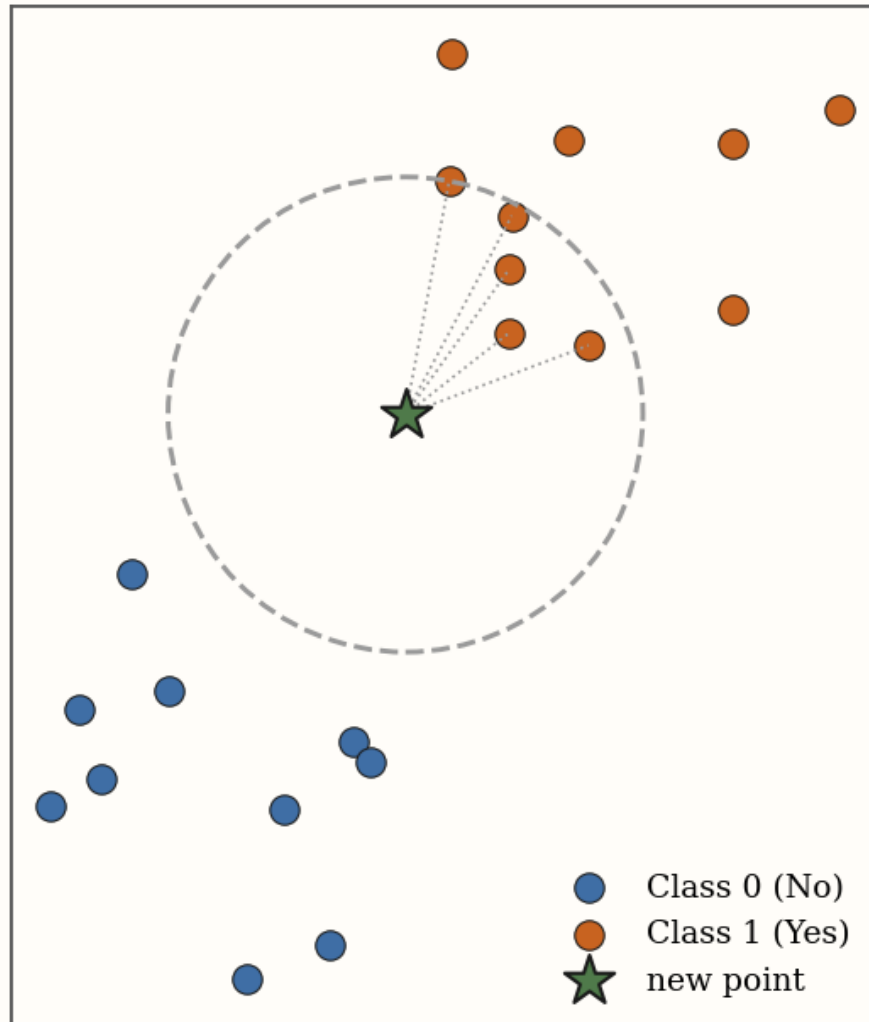
SAME GOLDEN RULE

Frequencies are learned in `fit()` from the **training data**, then simply mapped onto the test data in `transform()` — no leakage.

10 Training & Evaluating a KNN Classifier

With features scaled and categories encoded, we can train a model. **K-nearest neighbors (KNN)** is a distance-based classifier: to label a new point, it looks at its k closest training examples and takes a **majority vote**. Because it relies on distance, KNN is exactly the kind of algorithm that *needs* scaled inputs.

KNN with $k = 5$: the new point takes the majority class of its 5 nearest neighbours



With $k = 5$, the new point is classified by the most common class among its five nearest neighbors.

```
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train_encoded, y_train)

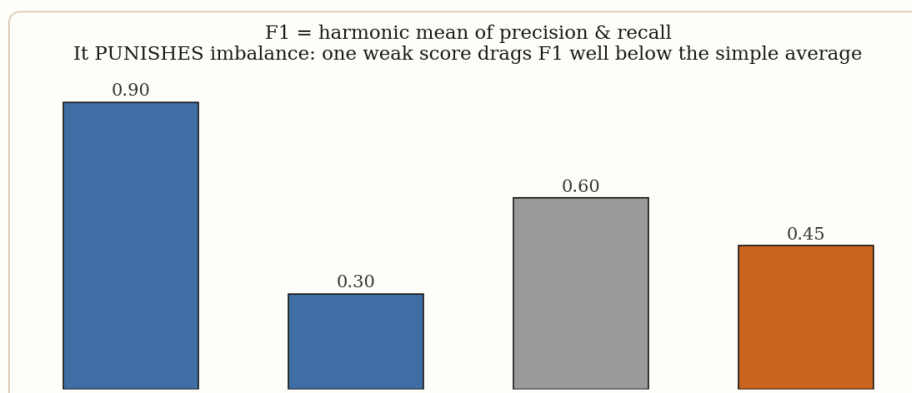
predictions = knn_model.predict(X_test_encoded)
predictions_proba = knn_model.predict_proba(X_test_encoded)

# Judge the model from several angles
print("Accuracy :", accuracy_score(y_test, predictions))           # 0.630
print("Precision:", precision_score(y_test, predictions, pos_label='Yes')) # 0.611
print("Recall   :", recall_score(y_test, predictions, pos_label='Yes'))  # 0.585
print("F1 Score :", f1_score(y_test, predictions, pos_label='Yes'))      # harmonic mean
print("AUC      :", roc_auc_score(y_test, predictions_proba[:, 1]))
```

Reading the scorecard

One number rarely tells the whole story, so we check several:

Metric	What it measures	Care most when...
Accuracy	Share of all predictions that are correct	Classes are balanced
Precision	Of predicted positives, how many are truly positive	False positives are costly
Recall	Of actual positives, how many you caught	False negatives are costly
F1 score	Harmonic mean of precision and recall	You need a balance of both
AUC	Ranking quality — how reliably it scores real positives above real negatives, across every threshold	You care about ranking / probabilities



The F1 score uses the harmonic mean, so a lopsided pair (high precision, low recall) is dragged well below a simple average — you can't fake it with one strong number.

REMEMBER

`predict()` returns the chosen class; `predict_proba()` returns the probability of each class. For binary problems that's two numbers per row — the chance of class 0 and class 1.

Quick Reference

The whole guide on one page.

Method	Formula / basis	Range or result	Outliers
Standardization	$(x - \text{mean}) / \text{std}$	mean 0, std 1	Sensitive
Min-max	$(x - \text{min}) / (\text{max} - \text{min})$	0 to 1	Very sensitive
Robust	$(x - \text{median}) / \text{IQR}$	centered on median	Resistant
Log	$\log(x)$	compressed, less skew	Pulled inward

Habit	Do this
Order of operations	Split data first , then scale
Training data	<code>fit_transform()</code> — learn <i>and</i> apply
Test data	<code>transform()</code> only — never re-fit
Categorical features	Encode (e.g. frequency encoding) so they're numeric
Distance / gradient models	KNN, SVM, neural nets, PCA → always scale
Tree-based models	Usually fine without scaling

ONE-LINE TAKEAWAY

Scaling puts every feature on equal footing; the golden rule is to learn the scale from training data alone.

Vocabulary & Concepts

Every term from this guide, in plain language (103 entries).

Accuracy	The proportion of predictions that match the actual class labels.
Accuracy score	A classification metric used to measure overall predictive correctness.
Area Under the ROC Curve (AUC)	A metric that measures how well a classifier separates classes across different classification thresholds.
BaseEstimator	A scikit-learn base class that supplies standard estimator behavior, including parameter handling.
Bessel's correction	An adjustment used when estimating sample variance or standard deviation; it explains small differences between pandas and scikit-learn calculations.
Biased model	A model whose learning is disproportionately influenced by certain features, such as features with larger numeric magnitudes.
Binary classification	A classification problem with two possible target classes.
Bounded data	Data with known or fixed lower and upper limits; often suitable for min-max scaling.
Categorical data	Data represented by discrete labels or groups rather than continuous numeric measurements.
Centering	Subtracting a feature's mean from each observation so the transformed data is centered around zero.
Class label	The category assigned to an observation by a classifier.
Data distribution	The pattern describing how values are spread; its shape helps determine the appropriate scaling method.
Data preprocessing	Preparing and transforming raw data before training a machine-learning model.
Data leakage	Improperly allowing test-set information to influence training; fitting a scaler on test data would cause leakage.
Distance-based method	An algorithm that depends on distances between observations and is therefore sensitive to feature scale; KNN is an example.
Dominant feature	A feature that exerts excessive influence because its values have a much larger magnitude than other features.
Exponential distribution	A distribution whose large values can often be compressed effectively with a logarithmic transformation.
F1 score	The harmonic mean of precision and recall, used when both false positives and false negatives matter.
Feature	An input variable used by a machine-learning model.
Feature engineering	Creating or transforming features to make data more useful for machine learning.
Feature matrix (X)	The table or array containing the model's input features.
Feature range	The lower and upper bounds chosen for transformed values, commonly 0 and 1 in min-max scaling.
Feature scaling	

	Transforming numeric features to comparable scales so that no feature dominates because of its units or magnitude.
fit()	A method that learns parameters from training data, such as category frequencies, means, standard deviations, minimums, or maximums.
fit_transform()	A method that learns transformation parameters from data and then transforms that same data.
Frequency encoding	Replacing each categorical label with its observed frequency or relative frequency in the training data.
Frequency mapping	A stored association between each category and the category's frequency.
Gaussian distribution	Another name for the normal distribution; standardization is especially useful for approximately Gaussian data.
Gaussian Naive Bayes	A classification algorithm that assumes numeric features follow Gaussian distributions.
Gradient boosting	An ensemble modeling technique identified in the document as a possible application for robustly scaled data.
Gradient descent	An optimization method used to train models; uneven feature scales can produce oscillation, slow convergence, or failure to converge.
Harmonic mean	The type of average used to combine precision and recall in the F1 score.
Inference	Using a trained model to produce predictions for new or test observations.
Interquartile range (IQR)	The 75th percentile minus the 25th percentile; robust scaling uses it as a scale measure resistant to outliers.
K-nearest neighbors (KNN)	A distance-based algorithm that classifies an observation according to nearby training examples.
KNeighborsClassifier	The scikit-learn implementation of a KNN classification model.
Lasso regression	A regularized regression method that is sensitive to feature scale and generally requires scaled inputs.
Label encoding	Converting categorical labels into numeric codes.
Linear discriminant analysis	A classification method that may benefit from standardized, approximately normal input data.
Linear regression	A model that assumes a linear relationship between variables; log scaling can make some exponential relationships more linear.
Linearity	The degree to which a relationship between variables can be represented by a straight line.
Log scaling	Applying a logarithmic transformation to compress large values, spread smaller values, reduce right skew, and lessen outlier influence.
Logarithmic transformation	The mathematical transformation used in log scaling.
Logistic regression	A classification method that commonly benefits from standardized features.
Machine learning model	A learned mathematical system that makes predictions or classifications from data.

Majority vote	The KNN decision rule that assigns the class most common among the selected nearest neighbors.
Matrix operations	Calculations on arrays of data that can become numerically unstable when features have extremely different scales.
MaxAbs scaling	Scaling each feature by its maximum absolute value.
Mean	The arithmetic average; standardization subtracts the training-feature mean from every value.
Median	The 50th percentile; robust scaling uses it as a center that is less affected by extreme values than the mean.
Min-max scaling	Rescaling a feature to a fixed interval, usually 0 to 1, using $(x - \text{minimum}) / (\text{maximum} - \text{minimum})$.
MinMaxScaler	The scikit-learn transformer that performs min-max scaling.
Model interpretability	The ability to understand how features contribute to predictions; common scaling can make comparisons easier.
Model performance	The quality of a model's predictions as measured with suitable evaluation metrics.
Multiclass classification	A classification problem with more than two possible classes.
Multimodal data	Data with multiple peaks; ordinary standardization may be a poor choice when the distribution departs substantially from normality.
Neural network	A machine-learning model that is highly sensitive to feature scale and commonly requires scaled inputs.
Non-Gaussian data	Data that does not follow a normal distribution; robust scaling may be appropriate.
Normalization	In this document, unit-vector scaling — rescaling each observation (row) so its vector length equals 1. Heads up: many ML texts use “normalization” more broadly to mean any rescaling (often min-max to 0–1), so always check which sense an author intends.
Normal distribution	A symmetric bell-shaped distribution characterized by its mean and standard deviation.
Numeric feature	A measurable input represented by numbers, such as age or income.
Numerical stability	Reliable computation without overflow, underflow, or excessive rounding effects.
One-hot encoding	A categorical encoding method that creates separate indicator columns for categories; it is related to, but distinct from, frequency encoding.
Optimization	The process of adjusting model parameters to minimize error or another objective.
Ordinal encoding	Converting ordered categories into numeric values that preserve their rank.
Outlier	An unusually extreme observation that can distort scaling parameters and model behavior.
Overflow	A numerical error that occurs when a computed value exceeds the representable numeric range.
Power transform	A nonlinear feature transformation used to change distribution shape and stabilize variance.
Precision	The proportion of predicted positive cases that are actually positive; especially important when false positives are costly.
Predictive probability	A model's estimated probability that an observation belongs to a particular class.

Principal component analysis (PCA)	A scale-sensitive dimensionality-reduction method that typically works with standardized features.
Probability estimate	The estimated likelihood of membership in each possible class, returned in scikit-learn by <code>predict_proba()</code> .
Pseudo-random seed	A fixed value, such as <code>random_state=42</code> , used to make randomized operations reproducible.
Range	The maximum value minus the minimum value.
Recall	The proportion of actual positive cases correctly detected; especially important when false negatives are costly.
Relative frequency	A category's count divided by the total number of observations.
Ridge regression	A regularized regression method that is sensitive to feature scale and generally uses standardized inputs.
Robust scaling	Centering with the median and scaling with the interquartile range to reduce sensitivity to outliers.
Robustness	A model or transformation's ability to remain reliable despite outliers or unusual observations.
Scaling parameter	A value learned from training data and used in transformation, such as a mean, standard deviation, minimum, maximum, median, or IQR.
scikit-learn	The Python machine-learning library used for splitting data, scaling features, building the KNN model, and calculating metrics.
Skewed distribution	An asymmetric distribution with a longer tail on one side.
Standard deviation	The square root of variance; standardization divides centered values by this quantity.
Standard normal distribution	A normal distribution with mean 0 and variance 1.
Standard scaling	Another name for standardization or Z-score scaling.
Standardization	Transforming values using $z = (x - \text{mean}) / \text{standard deviation}$ so the result has approximately zero mean and unit variance.
StandardScaler	The scikit-learn transformer that performs standardization.
Support vector machine (SVM)	A scale-sensitive machine-learning algorithm that commonly requires standardized features.
Target vector (y)	The array containing the desired class label or output for every observation.
Test data	Held-out observations used to evaluate a trained model; they must be transformed using parameters learned only from training data.
Test size	The proportion of a dataset reserved for testing, such as <code>test_size=0.2</code> .
Training data	The observations used to fit scalers, encoders, and the machine-learning model.
train_test_split()	A scikit-learn function that divides data into training and test subsets.
transform()	A method that applies already-learned transformation parameters to data without refitting them.

TransformerMixin	A scikit-learn mixin that supplies transformation-related behavior, including <code>fit_transform()</code> .
Tree-based model	A model based on decision trees; the document lists such models as possible users of robustly scaled data, although most tree models ordinarily do not require scaling.
Underflow	A numerical error that occurs when a computed value is too close to zero to be represented accurately.
Unit invariance	A property obtained when transformed values no longer depend on the original measurement units.
Unit variance	Variance equal to 1, a defining result of standardization.
Unit-vector scaling	Rescaling an observation so its vector length equals 1; also called normalization in the document.
Variance	The average squared dispersion around the mean; its square root is the standard deviation.
Z-score	The number of standard deviations that a value lies above or below the mean.
Z-score scaling	Another name for standardization.

Learn More

Curated, beginner-safe links to the official documentation behind everything above.

scikit-learn: Preprocessing data (user guide)

<https://scikit-learn.org/stable/modules/preprocessing.html>

The official overview of scaling, standardization, and normalization — the single best next read.

StandardScaler API reference

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

Every parameter of the Z-score scaler used in this guide, plus worked examples.

MinMaxScaler API reference

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>

How to set `feature_range` and read back `data_min_ / data_max_`.

RobustScaler API reference

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.RobustScaler.html>

The median + IQR scaler for data with outliers.

Compare the effect of different scalers

https://scikit-learn.org/stable/auto_examples/preprocessing/plot_all_scaling.html

A visual side-by-side of every scaler on real data with outliers — highly recommended.

KNeighborsClassifier API reference

<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

The KNN model, including `n_neighbors` and `predict_proba`.

Classification metrics (accuracy, precision, recall, F1, AUC)

https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics

Definitions and formulas for every score printed in the final demo.

pandas: value_counts()

https://pandas.pydata.org/docs/reference/api/pandas.Series.value_counts.html

The engine behind frequency encoding, including `normalize=True`.



Scaling Numeric Data for Machine Learning

A Qwik Guide · AI Foundations Series

© 2026 David Grunwald. All rights reserved.